

Lecture 4

**Cloud Computing & Load
Balancing, Consistency &
Concurrency Control**

COMP2211: Distributed Systems

Thiru Vasantam

Topic 5 continued: Cloud Computing and Load Balancing

Resource Allocation: Traditional V/S Modern Methods

Data Center architecture and uses have been changing

Traditional-Static

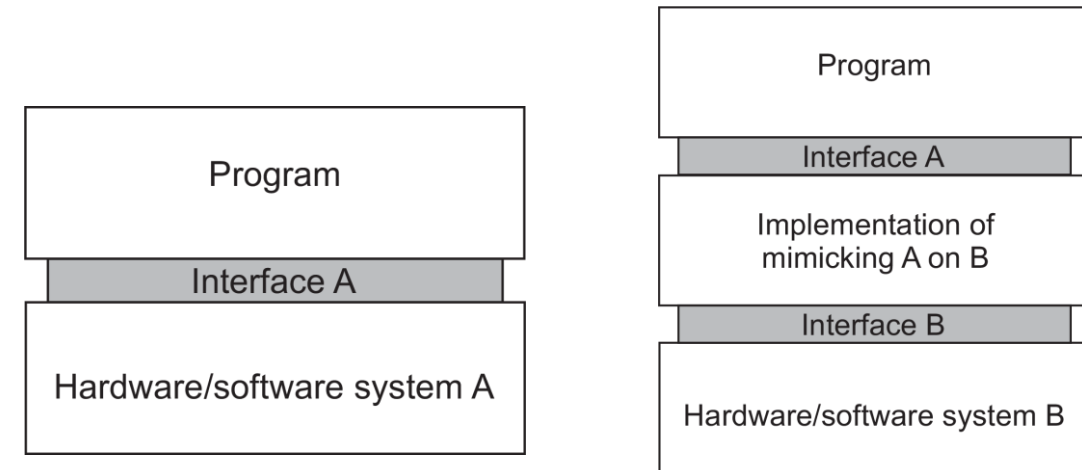
- Applications run on physical servers
- System administrators monitor and manually manage servers
- Use Storage Array Networks (SAN) or Network Attached Storage (NAS) to hold data

Modern -Dynamic, larger scale

- Run applications inside virtual machines
- Flexible mapping from virtual to physical resources
- Increased automation allows larger scale

Virtualization

- Virtualization is a technology that you can use to create virtual representations of servers and other physical machines. Virtual software mimics the functions of physical hardware to run multiple virtual machines simultaneously on a single physical machine.
- Virtualization is important, and has several advantages
 - Hardware **changes faster** than software (e.g. middleware and applications)
 - Ease of **portability** and code migration
 - **Isolation** of failing or attacked components
- **Ex:** Implementation of System A on System B using virtualization; A can be an old system, and B can be a new system



Ex: Software-Defined-Networking for Internet Switches

- In initial switches, the control plane (its software) was part of the hardware; its content depended on the hardware
 - Difficult to change configuration, routing/scheduling algorithms, topology, etc.
- They have separated the control plane from the hardware using software-defined networking (virtualization)
- You can learn more at:
<https://www.vmware.com/topics/software-defined-networking>

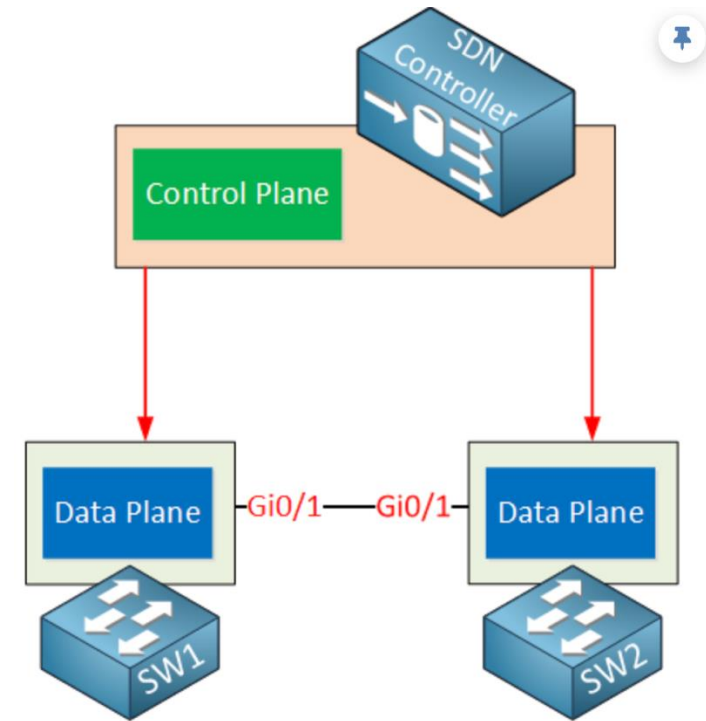


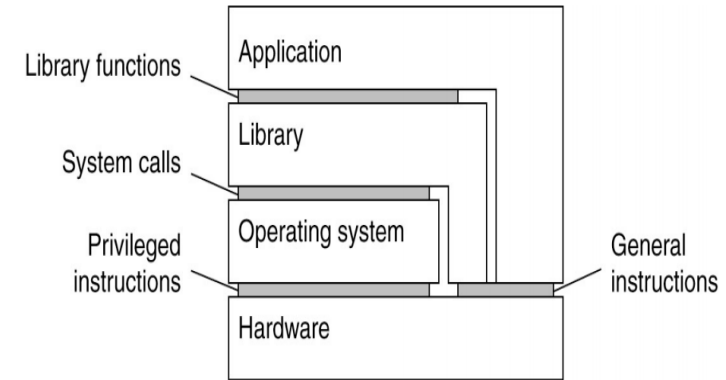
Image source: <https://networklessons.com/network-automation/introduction-to-sdn-software-defined-networking>

Mimicking Interfaces

- Four types of interfaces at three different levels

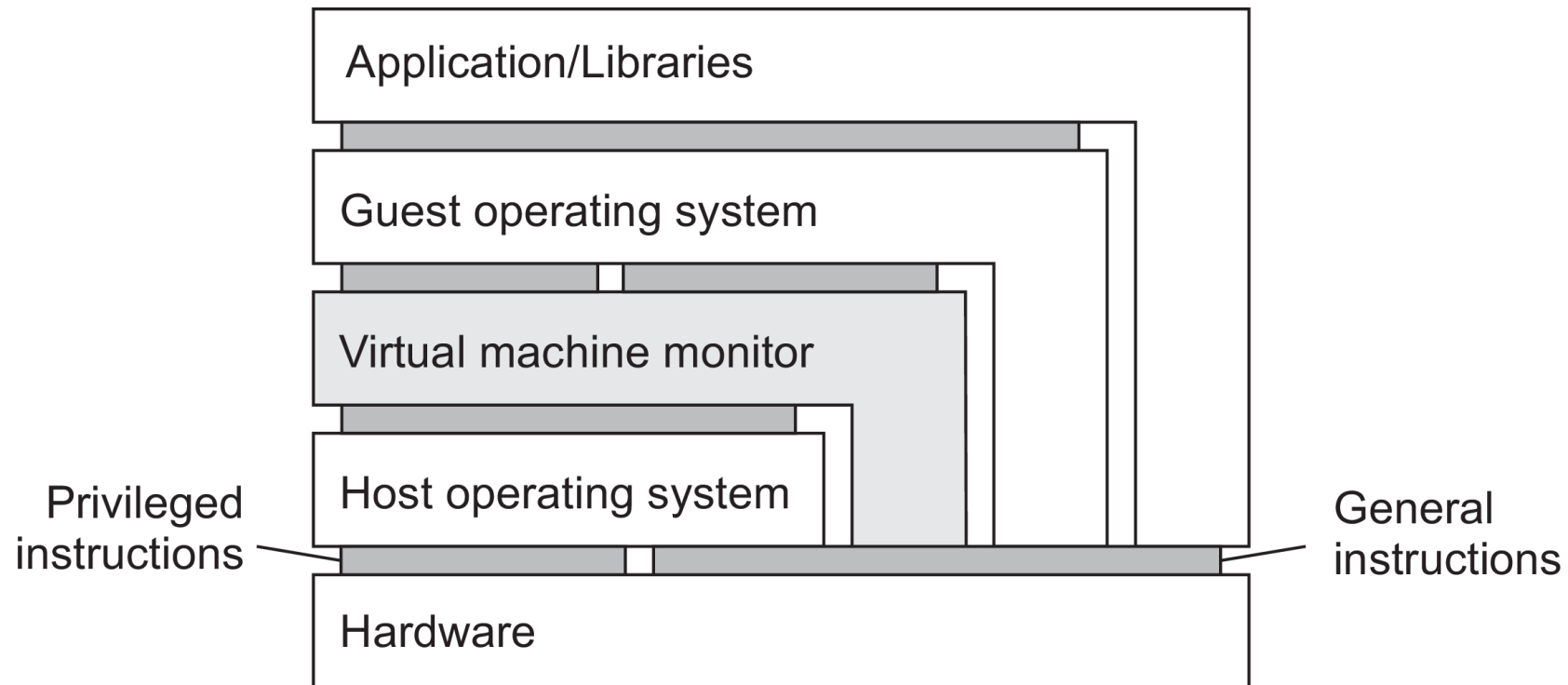
1. **Instruction set architecture**: the set of machine instructions, with two subsets:
 - Privileged instructions: allowed to be executed only by the operating system.
 - General instructions: can be executed by any program.
2. **System calls** are offered by an operating system.
3. **Library calls**, known as an **application programming interface** (API)

Depending on what is replaced /mimicked, we obtain different forms of virtualization



Source: Steen and Tannenbaum textbook

Example: OS Virtualization



*Source: Steen and
Tannenbaum textbook*

Virtual Machines (VMs)

- They are used for virtualization of resources like CPU, memory, etc., in cloud computing systems.
- **Virtual appliance:** pre-configured VM with OS/ apps pre-installed
 - Just download and run (no need to install/configure)
 - Software distribution using appliances
- Ex: **Multi-core CPUs**
 - Run multiple VMs on multi-core systems
 - Each VM assigned one or more vCPU
 - Mapping from vCPUs to physical CPUs

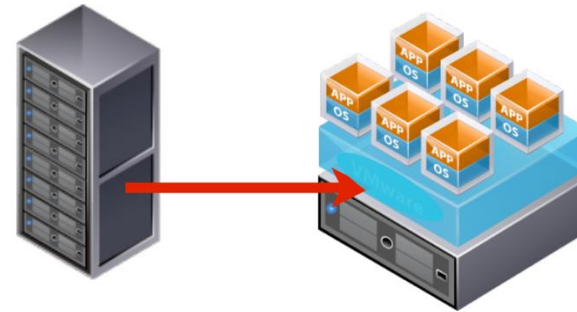
VMs in Data Centers

Virtual Servers (VMs)

- Consolidate servers
- Faster deployment, just implementing VM software which can be modified easily
- Easier maintenance

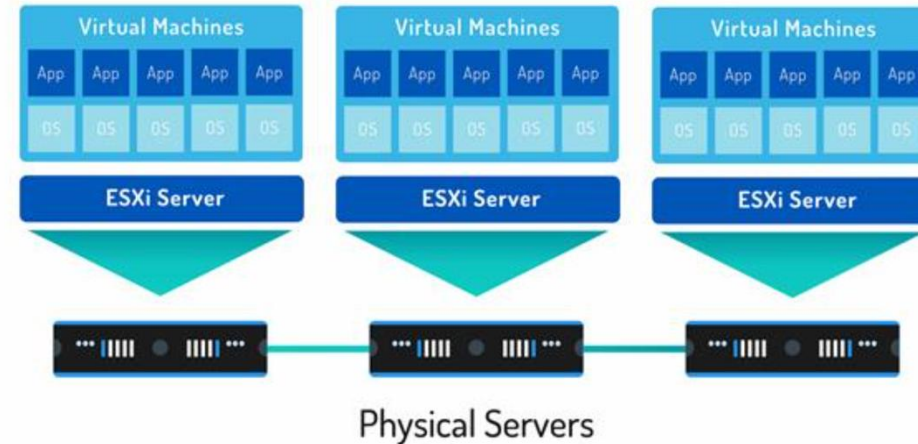
Key Points:

- Allows a server to be “sliced” into Virtual Machines
- VM has own OS/applications
- Rapidly adjust resource allocations
- VM migration within a LAN
- VM has a public IP address (to talk to any machine on the internet) and a private IP address (to talk to other VMs in the private network)



VMware Products

- The company VMware provides virtualization products.
- For example, their software product VMware ESXi allows VMs to access the resources of physical servers



Source:

<https://www.cloud4u.com/blog/what-is-vmware-esxi/>

Infrastructure-as-a-Service (IaaS) Cloud

- Instead of renting out a physical machine for the use of resources (like CPU, memory), a cloud provider will rent out a VM that may be sharing a physical machine with other customers, almost complete isolation between customers can be achieved
- **Amazon EC2: rents servers and storage to customers**

Uses virtualization to share each server for multiple customers

viewing 7/39 available instances

Instance name ▲	On-Demand hourly rate ▼	vCPU ▼	Memory ▼	Storage ▼	Network performance ▼
t4g.nano	\$0.0042	2	0.5 GiB	EBS Only	Up to 5 Gigabit
t4g.micro	\$0.0084	2	1 GiB	EBS Only	Up to 5 Gigabit
t4g.small	\$0.0168	2	2 GiB	EBS Only	Up to 5 Gigabit
t4g.medium	\$0.0336	2	4 GiB	EBS Only	Up to 5 Gigabit
t4g.large	\$0.0672	2	8 GiB	EBS Only	Up to 5 Gigabit
t4g.xlarge	\$0.1344	4	16 GiB	EBS Only	Up to 5 Gigabit
t4g.2xlarge	\$0.2688	8	32 GiB	EBS Only	Up to 5 Gigabit

<https://aws.amazon.com/ec2/pricing/on-demand/>

Load Balancing



"Here comes the boss"

Background

- Replication offers fault tolerance and better performance to distributed systems.
- However, such benefits may no longer be held if some replicated servers are overloaded while others are idle.

Load Balancing

Aim

Utilize servers efficiently

Load Distribution:

- Transfer load (tasks) from heavily loaded servers to idle or lightly loaded ones
 - [Load Sharing vs. Load Balancing](#)
- Load Sharing: reduce the likelihood of an *unshared state* (e.g., idle servers)
- Load Balancing: go a step further by attempting to equalize loads at all servers

Study and compare algorithms for load distribution

Focus: load sharing → improve response time

Factors to Consider

- Heterogeneity in server capabilities and service requirements of jobs, knowledge of the system state (exact state/rough estimates/statistical parameters)
- Objective of the system
 - Minimise delay
 - Minimise energy consumption
 - Maximise the chance of accepting a request for service (e.g., allocating VMs in IaaS clouds)

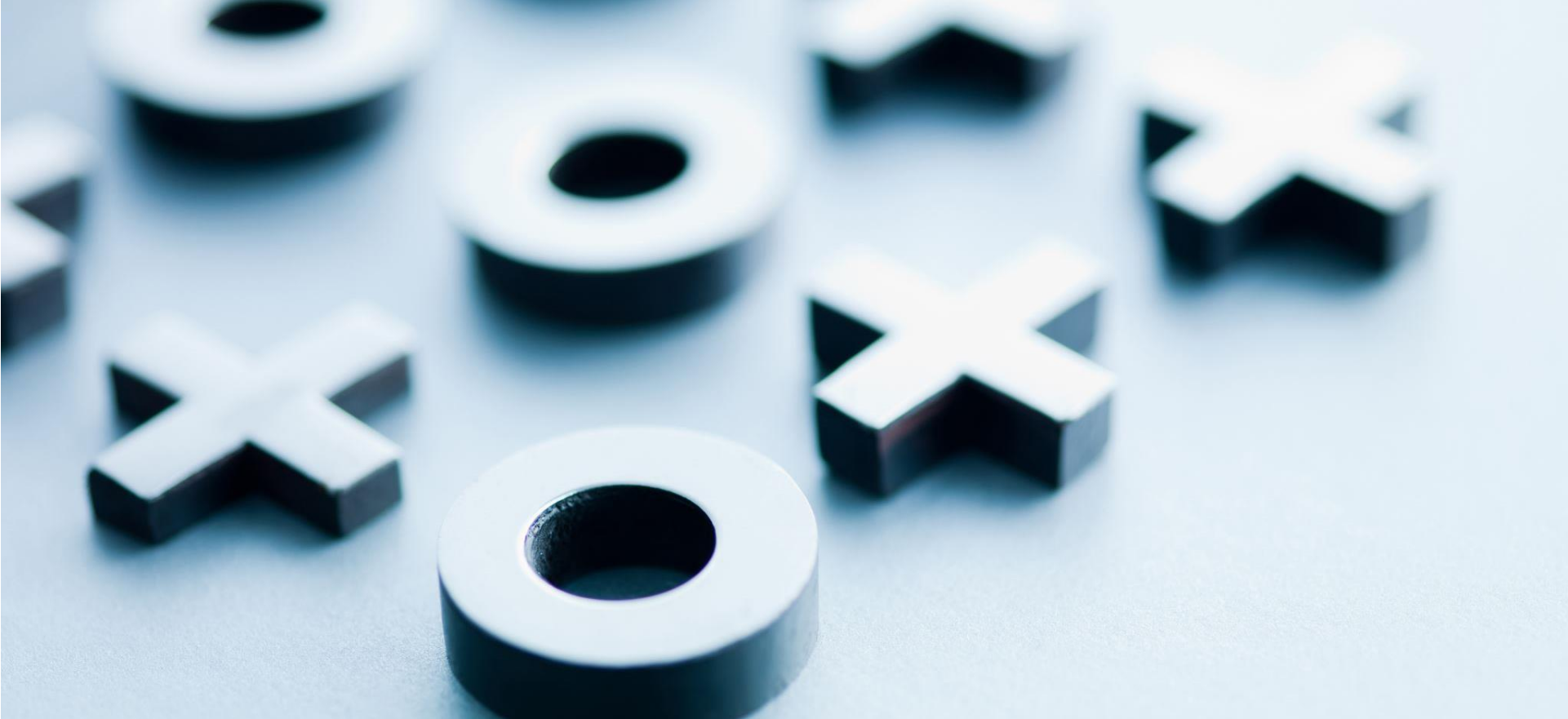
How to Approach Load Distribution

- ***Data Distribution (from tasks point of view)***

- To distribute incoming tasks to suitable servers
- To re-distribute ongoing tasks among servers

- ***Server Allocation (from servers point of view)***

- To determine suitable servers to receive / proactively distribute tasks (individual server perspective)
- To organise servers to work collaboratively for load sharing (collaborative server perspective)

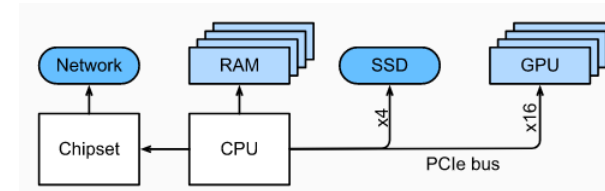
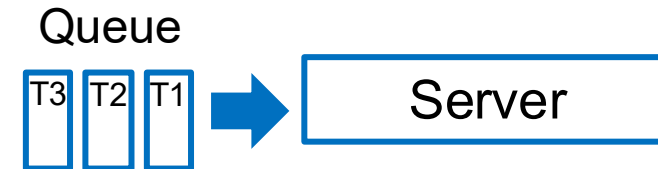


Workload and Simple Task Transfer

Where the workload comes from?

- Load Estimation

1. Queue length of waiting tasks: proportional to task response time
2. CPU (and GPU) utilization
3. Storage read/write bandwidth utilization
4. Network bandwidth utilization
5. Application dependent factors:
#transactions, #users,
#media content to access, etc.

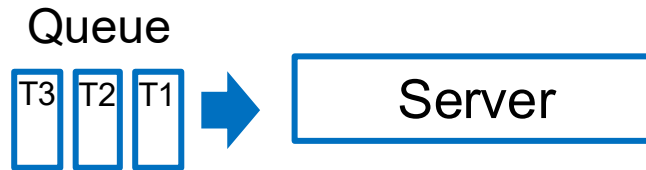


- Load Information Collection

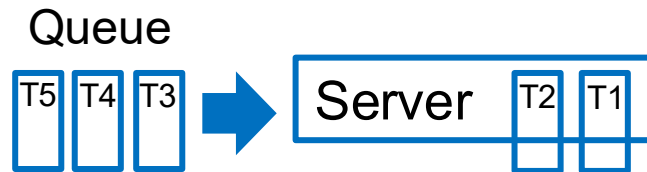
- Central coordinator: collects server load information centrally and globally
- Local approach: a server locally collects load information of neighbouring servers

Simple Task Transfer

- **Non-preemptive task transfers:** transfer tasks that have not started executing.
 - Transfer only the request (or task) without processing states
 - Good for load sharing but difficult for load balancing



- **Preemptive task transfers:** transfer tasks that are partially executed.
 - Expensive: involves collection and transmission of task states, e.g. virtual memory image, process control block, IO buffers, file pointers, timers, ...

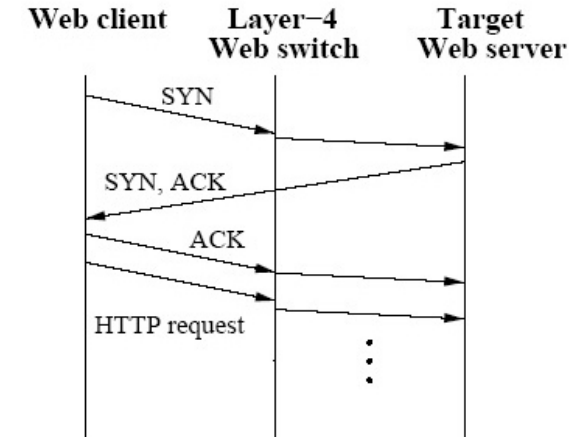


Routing Mechanisms based on network packet content

Two Types:

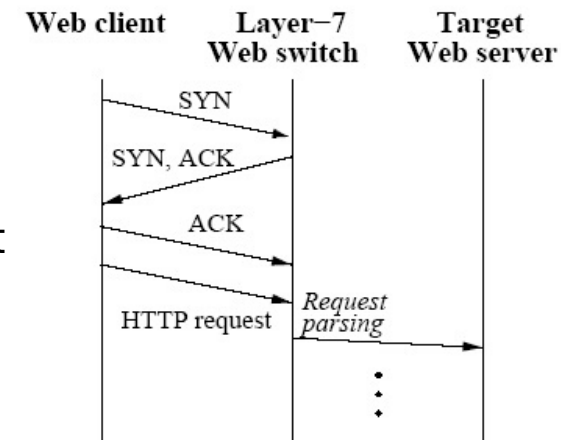
Layer-4 Routing

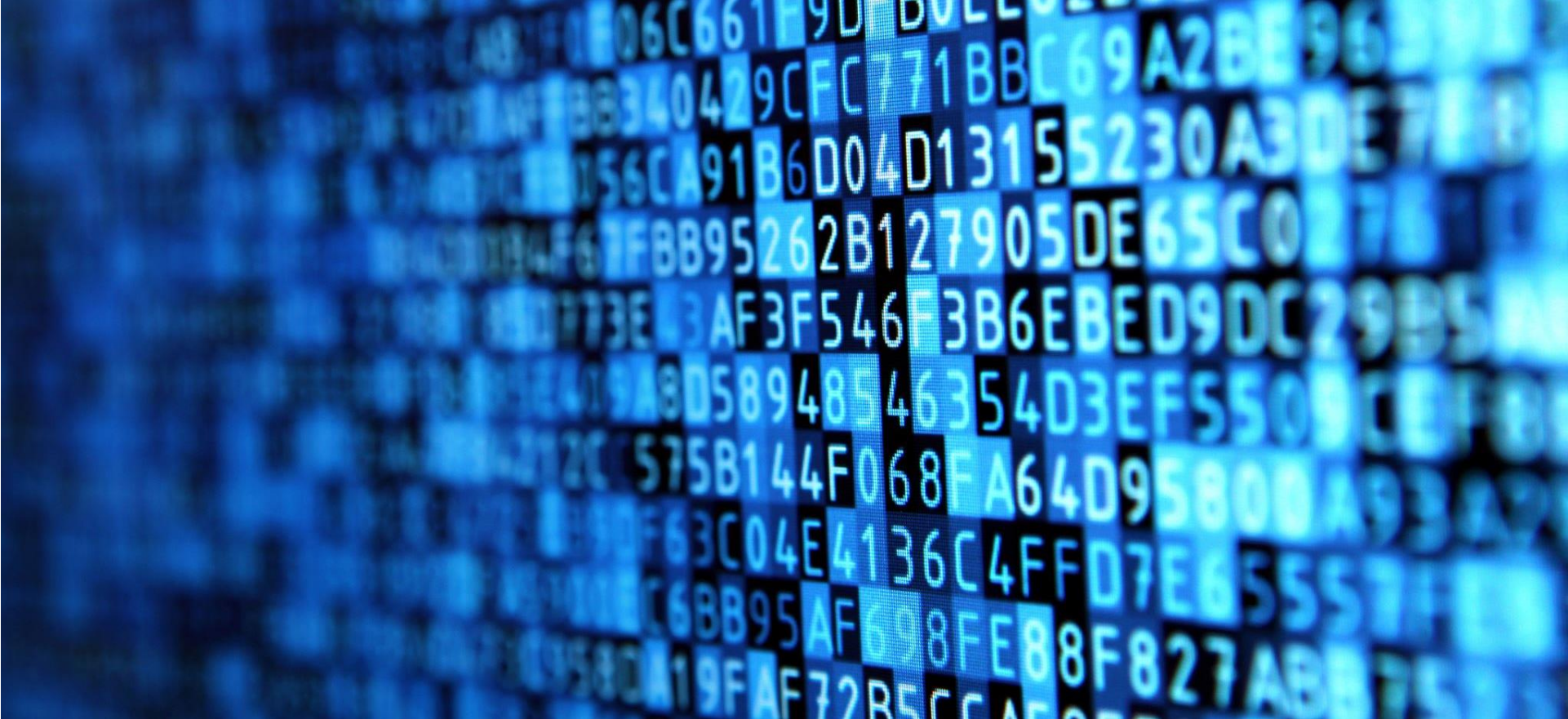
- Determines the target server without referring the message / request content (when TCP SYN packet is received).
- *Content-blind routing*: server selection is purely based on information from the IP header.



Layer-7 Routing

- Examine a request at the application level and select a server accordingly.
- Can support sophisticated dispatching policies, but may induce significant latency (also called *Content-aware routing*)



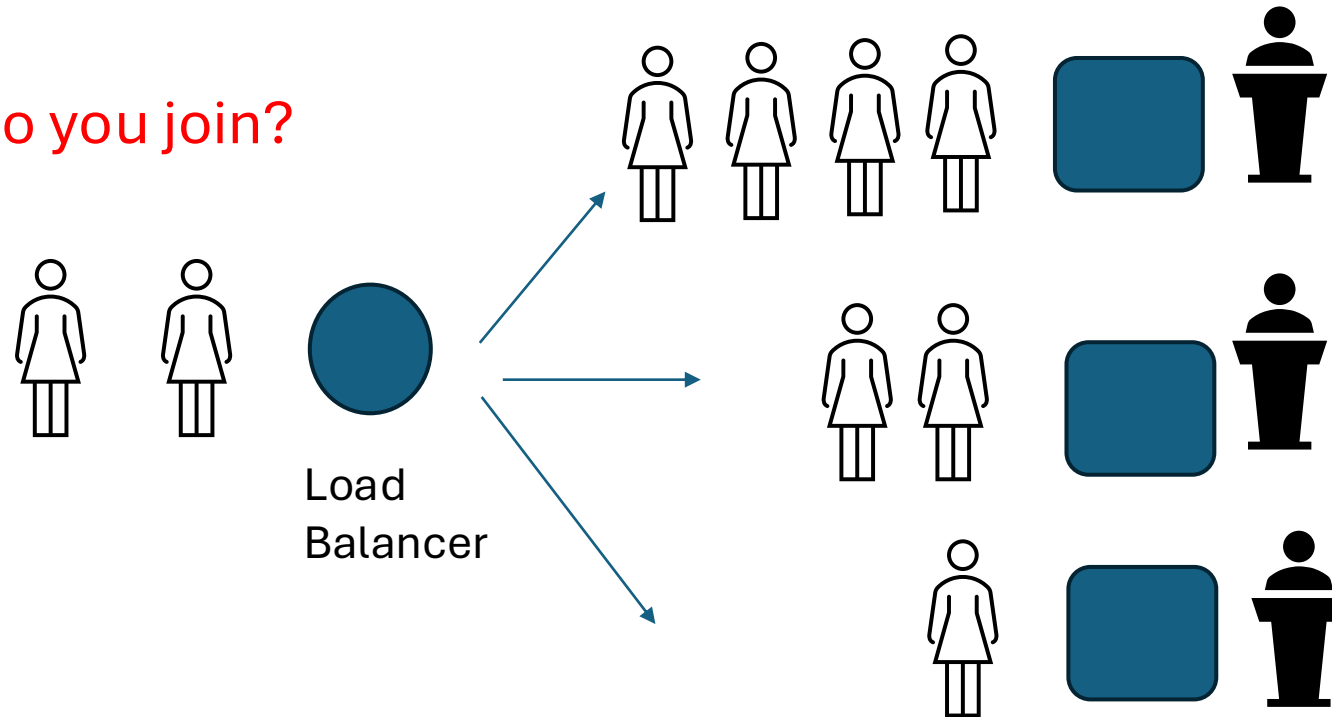


Load Distribution Algorithms

Load Balancing

Aim: Balance the load across servers to achieve small delays

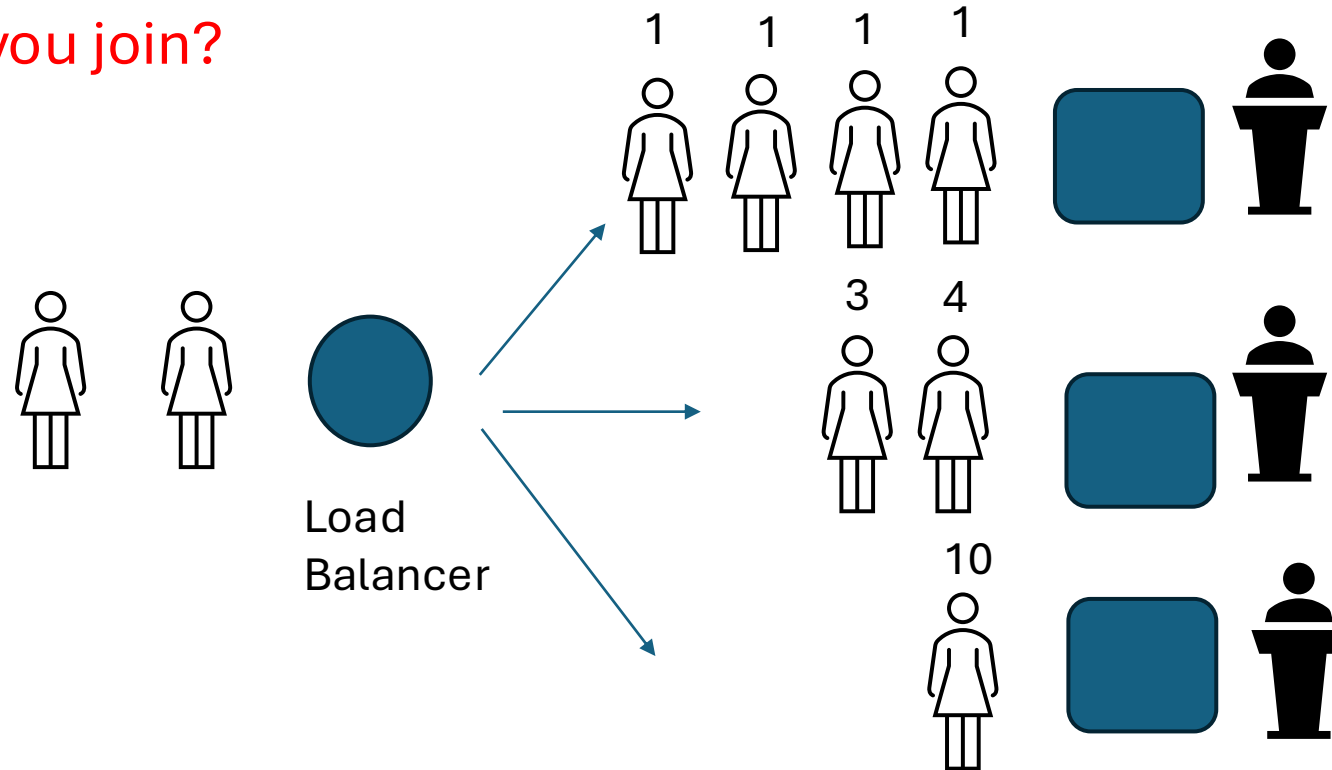
- Where do you join?



Join-the-Shortest-Queue (JSQ) policy: assign the job to the server with the shortest queue size

Load Balancing

Where do you join?



- **Least-Workload policy:** Send the job to the server with the least workload (time to finish currently assigned jobs); server 3 in the figure

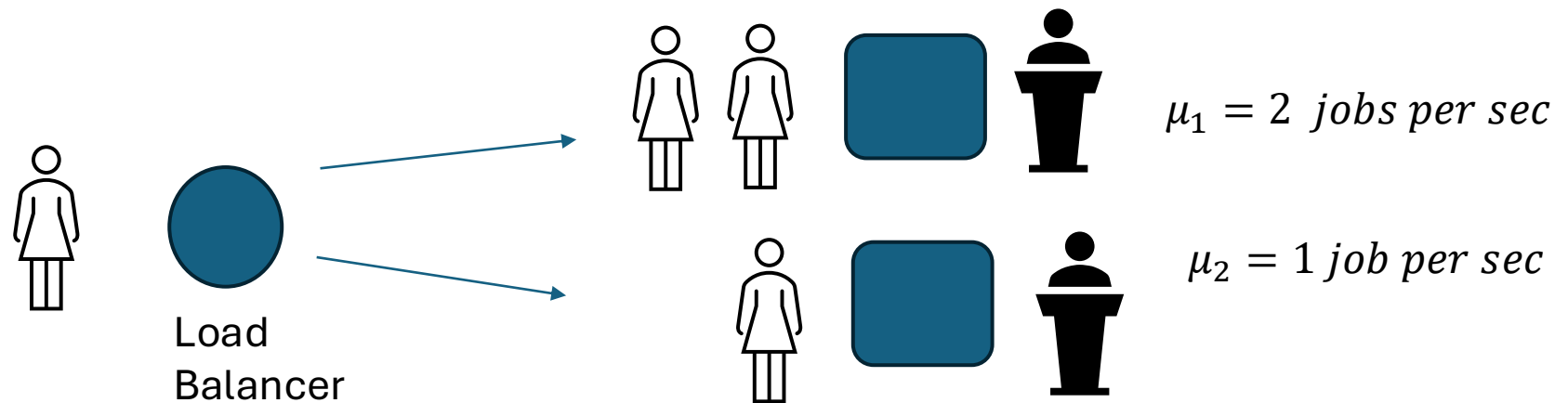
How to estimate the workload of servers?

- **Exact processing times:** Knowing the exact processing time (service-time) of every job and the remaining service-time of ongoing jobs, this is not always possible
- **Probability distributions:** Knowing the probability distributions of service times, we can estimate them from past data
- **Average service-times:** Knowing the average service-time of jobs, we can estimate these from the past data
- **Queue sizes:** Knowing the queue sizes of servers/applications

Major Approaches for Load Distribution

- **Static:** fixed policy, and the scheduling decision does not depend on the current state of the system

- Decisions are hard-coded into an algorithm, **Ex:** Round robin, allocations with some fixed probabilities
- Simple in implementation
- A priori knowledge of the system is required about servers and jobs
- **Ex:**



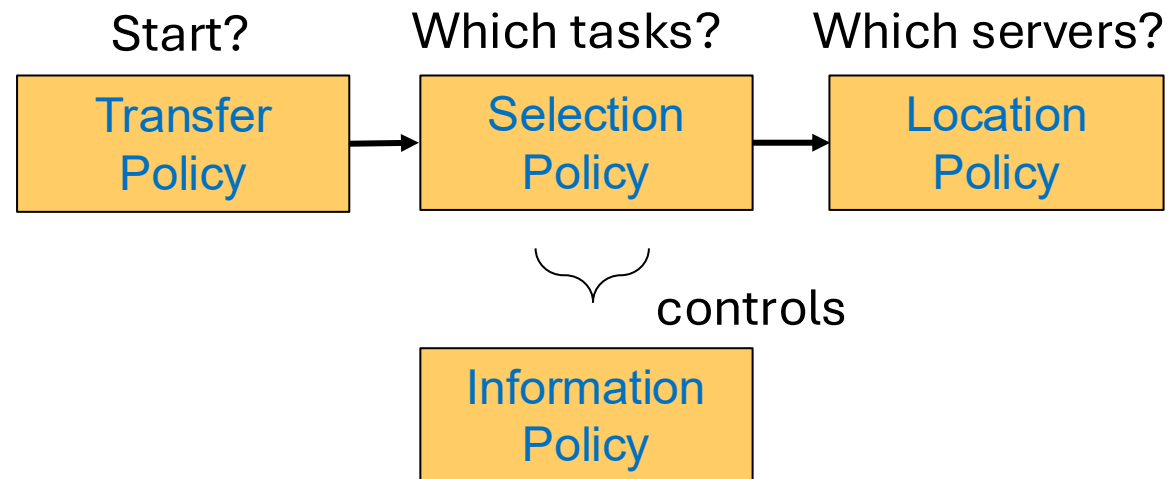
- Assign an incoming job to server 1 with probability $\frac{\mu_1}{\mu_1 + \mu_2}$.
- For this example, the Round robin policy is not a good policy; it results in higher delays as this policy allocates equal number of jobs to both servers.

Major Approaches for Load Distribution

- **Dynamic:** fixed policy, and the scheduling decision depends on the current state of the system, Ex: workload of servers
 - Make a decision during runtime based on system states: e.g. queue length (JSQ), processor utilisation
 - Correctness of load distribution depends on the timeliness of states collected (central vs. local coordinator)
- **Adaptive:** change policies dynamically
 - Enhance the dynamic approach by allowing:
 - Making choices of decision algorithms and the frequency of collecting load information based on system states

Construct a Load Distribution Algorithm

- Four main components
 - Include Transfer, Selection, Location, and Information policies
 - Define what information is required to collect and maintain supporting decision-making, and what procedures are required to follow for distributing workload



Transfer and Selection Policies

Transfer Policy

Decide whether a **server** needs to transfer tasks

- **Thresholds:** number of tasks, processor utilization, etc.
- **Role:** a server becomes sender/receiver when its load over/under a threshold
- **Issue:** Sensitive to time duration of a task transfer

Selection Policy

Determines which **task(s)** to transfer

- Tasks cause a server overloaded
- Estimated **task execution time**,
- Or **server response time** improvement
- Minimize **location-dependent** system calls made by the selected task

Location and Information Policies

Location Policy

Decide which **receiving server** for transferring a task

- *Polling* is generally used to find lightly loaded server
- can be done serially or in parallel (using multicast)

Information Policy

Decide when, where and what information to collect

- *Demand-driven*: a server collects the state of other servers only when it becomes either a sender or receiver
- *Periodic*: servers exchange load information periodically
- *State-change-driven*: servers disseminate state information whenever their state changes by a certain degree

When Does Load Distribution Help?

- What is the probability that at least one server is idle and one job is waiting?
 - High utilisation (all servers are busy) => little benefit
 - Low utilisation (all servers are lightly loaded, almost idle) => rarely jobs are waiting
- This probability is high for moderate system utilisation
 - Potential for performance improvement
 - Distributed scheduling (aka load balancing) is useful



Load Distributing Algorithms

Basic:

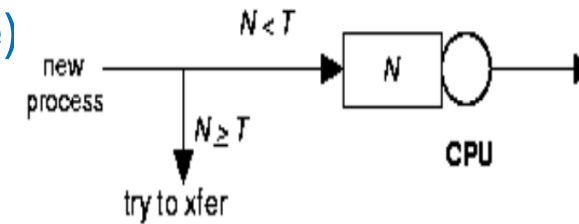
- **Sender-initiated:** distribution initiated by an overloaded server.
- **Receiver-initiated:** distribution initiated by lightly loaded servers.

Extensions:

- **Symmetric:** initiated by both senders and receivers. Has advantages and disadvantages of both the approaches.
- **Adaptive:** sensitive to state of the system.

Sender-Initiated (Example Design)

- **Transfer Policy:** Use thresholds.
 - **Sender**, if the queue length exceeds threshold T .
 - **The receiver**, if accepting a task, will not make the queue length exceed T .
- **Selection Policy:** Only newly arrived tasks (**non-preemptive**)
- **Location Policy (identify receiver):**
 - **Random:** Task transferred to a server at random.
 - No need for state collection
 - Unnecessary task transfers may occur
 - **Threshold:** poll a server to determine whether it is a receiver. The receiver must accept the task irrespective of its state when it (task) actually arrives.
 - Set up *PollLimit*, to reduce overhead
 - **Shortest:** Poll servers. Select the receiver with the shortest task queue length
- **Information Policy:** demand-driven.



Issues with Sender-Initiated Algorithm

- **Stability:** can become unstable at high loads.
 - At high loads, it may become difficult for senders to find receivers.
 - Also, the number of senders increases under high system load, thereby increasing polling activity.
 - Polling activity may make the system unstable at high loads.

Receiver-initiated

- **Transfer Policy:** Uses thresholds. If the queue length is below T , identify the server as the receiver, and those with a length above T as senders.
- **Selection Policy:** Newly arrived tasks (higher priority) + Preemptive task transfer (lower priority)
- **Location Policy:** Polling
 - Poll a random server, transferring a task when queue length $>$ threshold (i.e. sender)
 - Repeat the process to attempt transferring tasks from another server until **PollLimit** is reached
 - Poll a number of servers and choose the highly loaded one
- **Information policy:** demand-driven
- **Stability:** “Not unstable” since there are lightly loaded systems that have initiated the algorithm

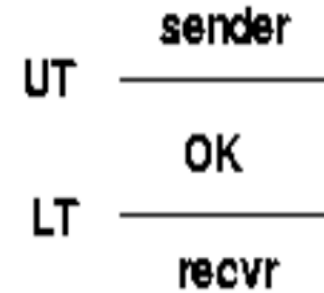
Receiver-initiated

- **Drawback:**

- Polling initiated by the receiver suggests it is difficult to find senders for new tasks.
- **Reason: systems try to schedule tasks as and when they arrive.**
- Effect: receiver-initiated approach might result in **preemptive transfers**. Hence, transferring costs are higher.
- Contrast: (Sender-initiated: transfer costs are low as new jobs are transferred, and so there is no need for transferring task states.)

Symmetric

- Senders search for receivers and vice versa.
- **Low loads:** senders can easily find receivers.
High loads: receivers can easily find senders.
- May have disadvantages of both:
 - Polling at high loads can make the system unstable.
 - Receiver-initiated task transfers can be preemptive and so expensive.



Implementation Methods:

- **Simple algorithm:** combine the previous two approaches.
- **Optimisation:** Adjust the scope of server search, e.g., local neighbouring servers

Adaptive Algorithm

Main Idea

- **Aim:** *Limit Sender's polling actions at high load to avoid instability.*
- Each server proactively keeps track of the status of all servers and polls adaptively



At each server

The process

- Each server maintains three lists. All servers are assumed to be receivers (i.e., not overloaded) initially
- **Location policy at sender:**
 - **Sender** polls the head of the receiver list
 - **Polled server** puts the sender at the head of its sender list, and informs the sender whether it is a receiver, a sender, or an OK server.
 - If the polled server is still a receiver, the new task is transferred to it.
 - Else sender updates lists and polls the next potential receiver.
 - **Stop when the receiver list finishes:** If this polling process fails to identify a receiver, the task can still be transferred during a **receiver-initiated dialogue**.

Adaptive Algorithm (cont.)

- **Location policy at receiver:**

- Receivers obtain tasks from **potential senders**. Lists are scanned in the following order.

* Spend more effort at a receiver to identify a sender

- Head to tail in senders list (**most up-to-date info used**), tail to head in OK list (**least up-to-date used**), tail to head in receiver list.
- Least up-to-date used in the hope that status might have changed.
- **Transfer a task if a sender is found.**



- If the server is not a sender, both the polled server and receiver update each other's status.
- Polling process stops if a **sender is found** or a **static PollLimit** is reached.

Discussions of Adaptive Algorithm

- At **high loads, sender-initiated polling gradually reduces** as servers get removed from receiver list (and become senders).
 - Whereas at low loads, sender will generally find some receiver.
 - At **high loads, receiver-initiated works and can find a sender.**
 - At low loads, receiver may not find senders, but that does not affect the performance.
- * Algorithm dynamically becomes sender-initiated at low loads and receiver-initiated at high loads.
- Hence, **algorithm is stable** and can use non-preemptive transfers at low loads (sender initiated).

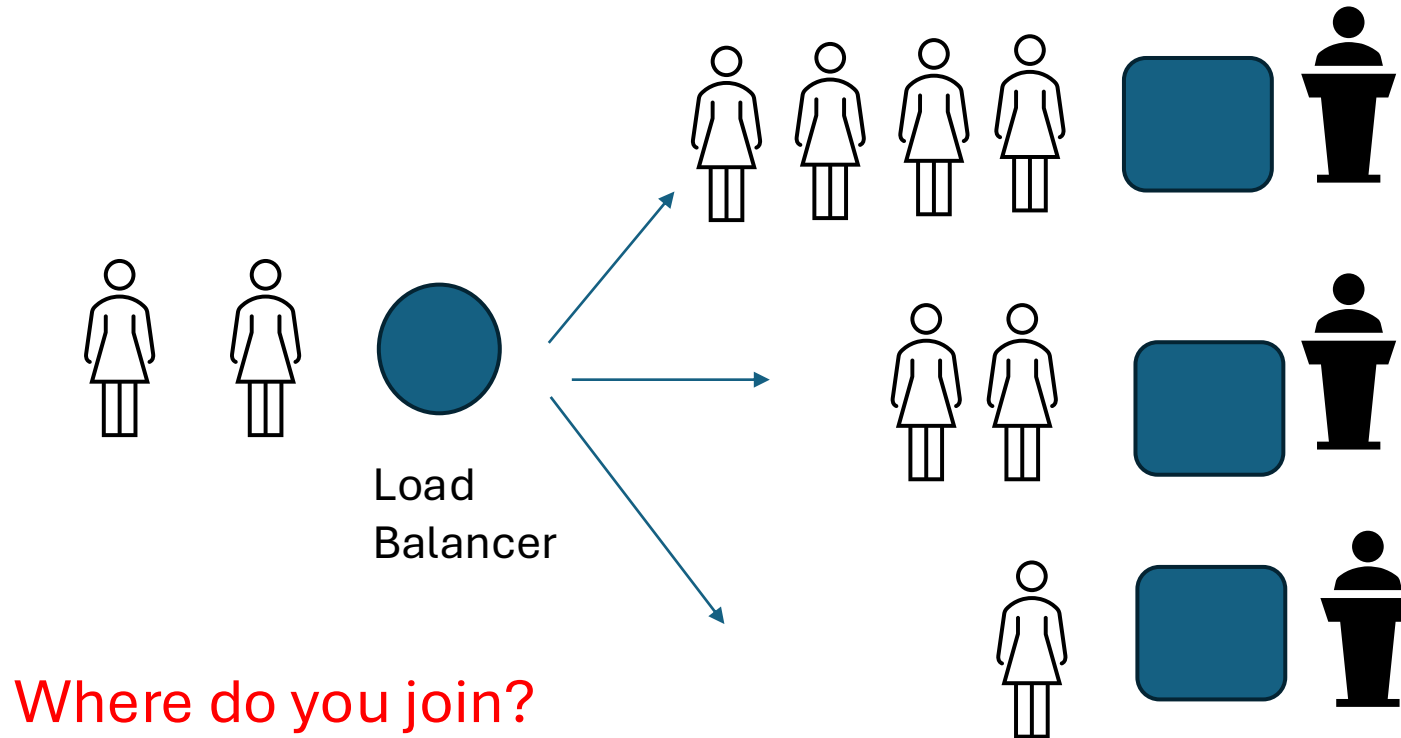
Selecting an Algorithm

- If a system never gets highly loaded, sender-initiated algorithms work better.
- Receiver-initiated algorithms better for high loads.
- Widely fluctuating loads: symmetric algorithms.
- Widely fluctuating loads + high migration cost for preemptive transfers: sender-initiated algorithms.
- Heterogeneous work arrival: adaptive algorithms.

Load Balancing Algorithms for Modern Data Centres

Load Balancing

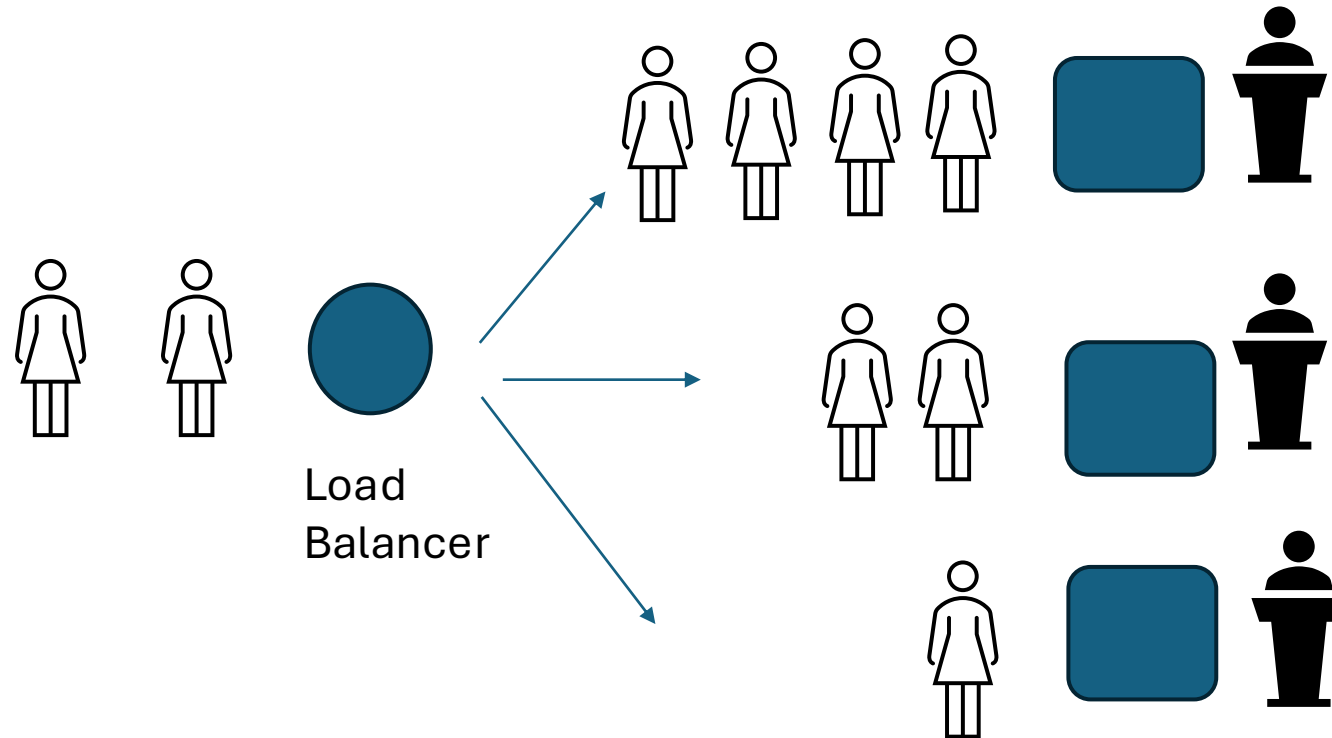
- What if the number of servers (N) is large, say 10000?



Power-of-d load balancing policy: assign the job to the server with the least queue size out of 'd' servers chosen uniformly at random.

Load Balancing

- What if the number of servers (N) is large, say 10000?



There is a significant performance gain (i.e., a reduction in delays) even for $d=2$ under the Power-of- d policy over the random routing policy (which corresponds to $d=1$)

The Power-of-d Choices

- The rate of incoming requests is λ
- The rate of service is $\mu = 1 \text{ job per sec}$
- It is very difficult to analyse computer systems with a small number of servers, such as 3 or 4, because of the complex dependencies introduced by the power-of-d policy.
- However, when the number of servers is large (>100), accurate analysis can be performed using mean-field techniques (a method typically used in statistical physics).

TABLE 1
Average Time in the Supermarket Model: 100 Queues

Choices	λ	Simulation	Prediction	Rel. Error (%)
2	0.50	1.2673	1.2657	0.1289
	0.70	1.6202	1.6145	0.3571
	0.80	1.9585	1.9475	0.5742
	0.90	2.6454	2.6141	1.1981
	0.95	3.4610	3.3830	2.3028
	0.99	5.9275	5.4320	9.1227
3	0.50	1.1277	1.1252	0.2146
	0.70	1.3634	1.3568	0.4858
	0.80	1.5940	1.5809	0.8314
	0.90	2.0614	2.0279	1.6533
	0.95	2.6137	2.5351	3.1002
	0.99	4.4080	3.8578	14.2607
5	0.50	1.0340	1.0312	0.2637
	0.70	1.1766	1.1681	0.7250
	0.80	1.3419	1.3289	0.9789
	0.90	1.6714	1.6329	2.3564
	0.95	2.0730	1.9888	4.2363
	0.99	3.4728	2.9017	19.6825

Source: Mitzenmacher 2001

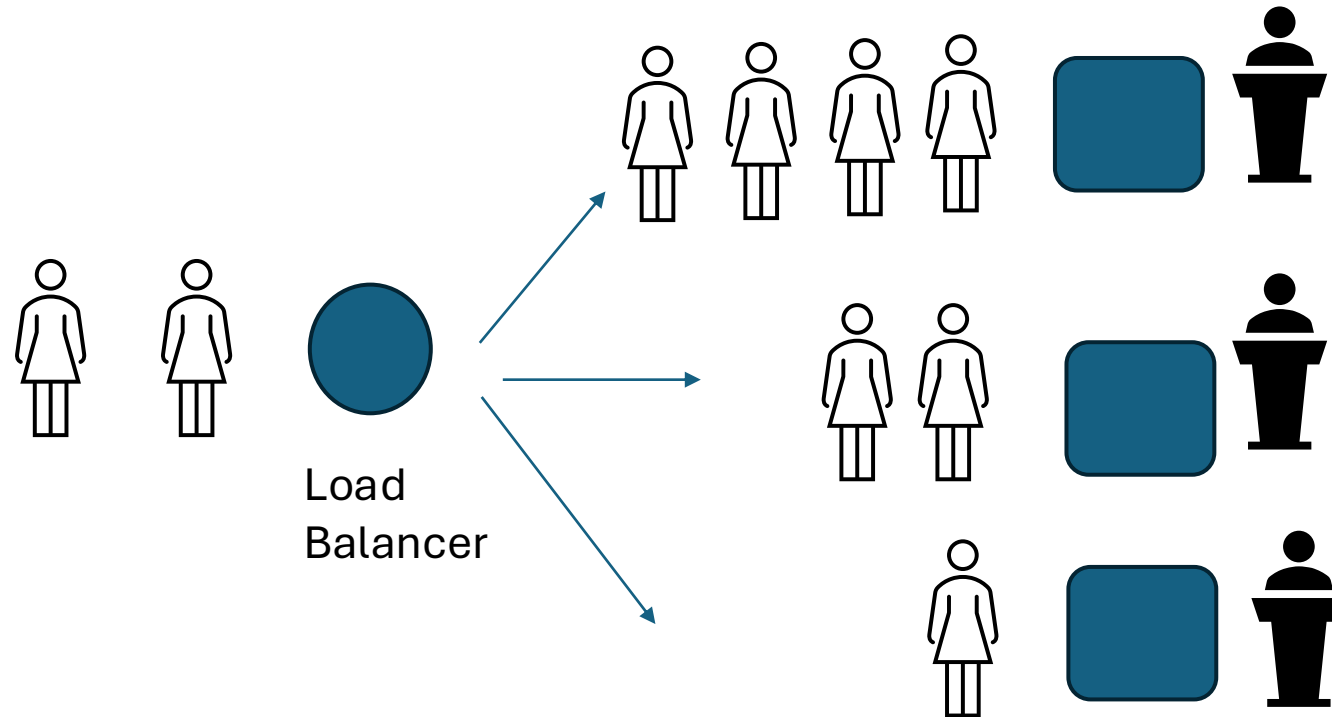
Implementations of Power-of-d in Practical Systems

Look at

- **For YouTube:** chrome-extension://efaidnbmnnnibpcajpcglclefindmkaj/https://www.usenix.org/system/files/nsdi24-wydrowski.pdf
- <https://medium.com/the-intuition-project/load-balancing-the-intuition-behind-the-power-of-two-random-choices-6de2e139ac2f>
- <https://www.nginx.com/blog/nginx-power-of-two-choices-load-balancing-algorithm/>

Load Balancing

- Can you think of any server-initiated algorithm?

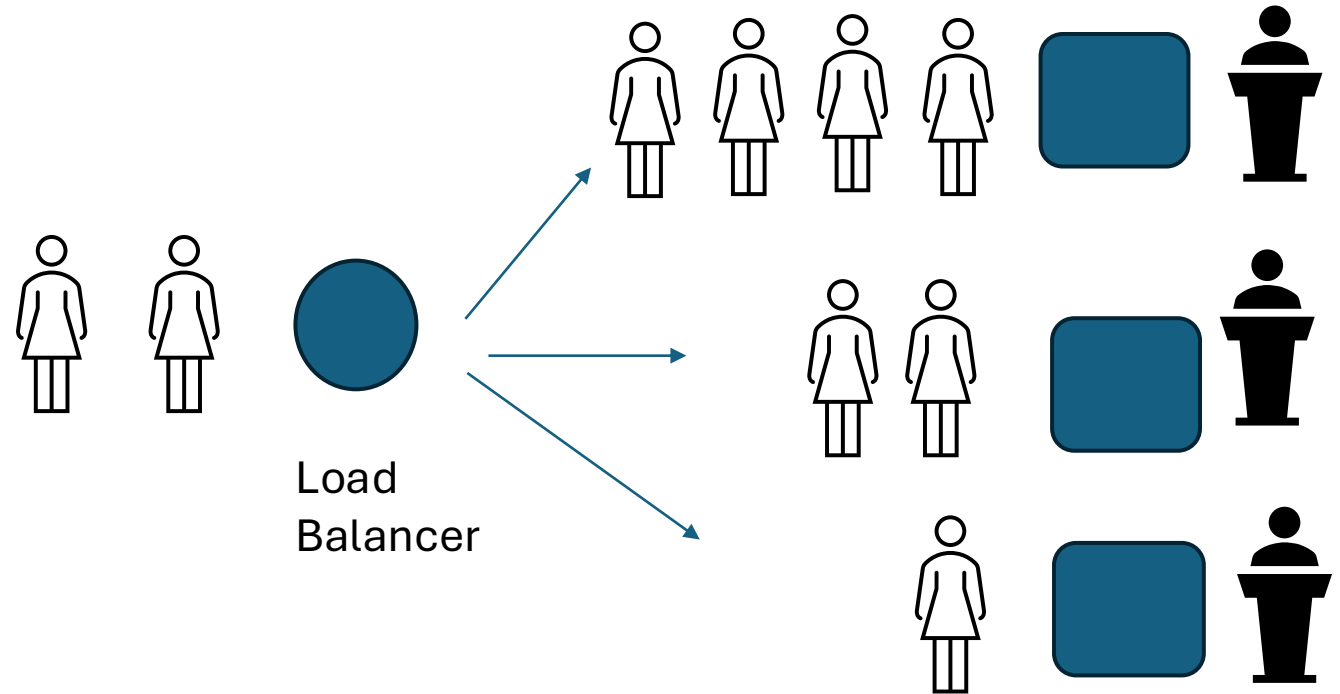


Load Balancing

- Can you think of any server-initiated algorithm?

Join-the-Idle-Queue (JIQ) policy:

- Servers inform the load balancer whenever they become **idle**.
- The load balancer then assigns an incoming job to one of the idle servers (from the list).
- If there is no idle server, then the job is assigned to a randomly selected server.
- It was observed that most of the time, at least one idle server is available to process incoming jobs.



Challenge: Maintaining the correct list of idle servers is challenging

Yi Lu, Qiaomin Xie, Gabriel Kliot, Alan Geller, James R. Larus, and Albert Greenberg. 2011. "Join-Idle-Queue: A novel load balancing algorithm for dynamically scalable web services," *Perform. Eval.* 68, 11 (November, 2011), 1056–1071. ([from Microsoft research](#))

Job Processing at Servers

Choosing the right scheduling policy based on the application

- Delay-sensitive applications such as YouTube requests, video conferencing, and social networks require immediate server allocation.
 - These jobs are typically processed using a **Processor-Sharing** discipline, where tasks are handled in a round-robin manner. Each job receives a small time slice (e.g., 1 ms) per round until completion. Because these time intervals are extremely short, users perceive the service as continuous.
- In contrast, applications like email sending or file downloads are less time-sensitive. Their jobs can be processed using a **First-Come-First-Served (FCFS)** scheduling algorithm.

Reference

- Chapter 3 of
Distributed Systems, 4th Edition, Maarten van Steen and Andrew Tannenbaum.
- Niranjan G. Shivaratri , Phillip Krueger , Mukesh Singhal, Load Distributing for Locally Distributed Systems, Computer, v.25 n.12, p.33-44, December 1992
- V. Cardellini and E. Casalicchio, “The State of the Art in Locally Distributed Web-Server Systems,” *ACM Computing Surveys*, **34**(2):170-177, June 2002.



COMP2211

Distributed Systems

Topic 6: Concurrent Events and Consistency Control

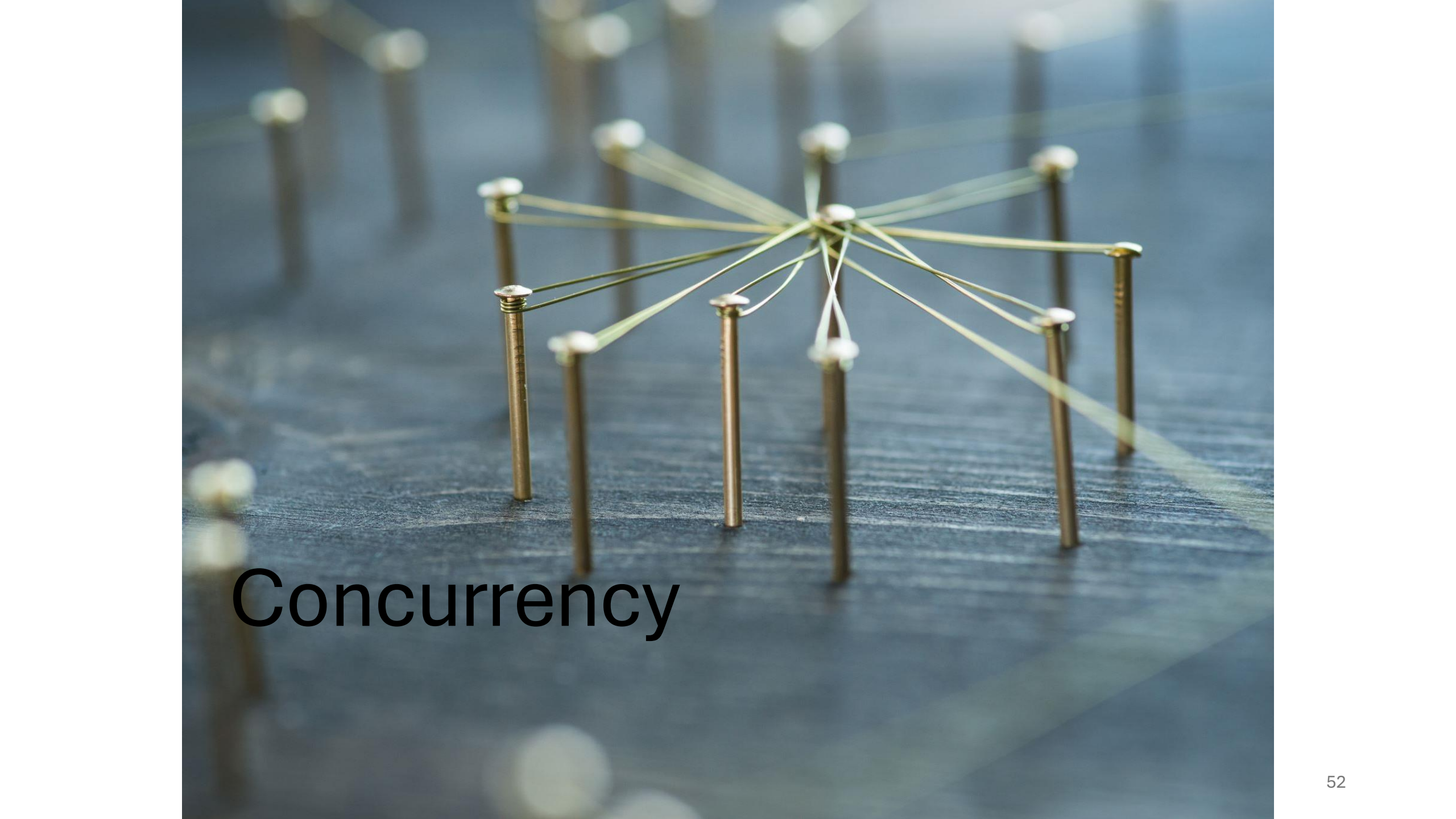
Thiru Vasantam

This Topic

Aim

Study the characteristics of concurrent events in distributed systems

Study mechanisms to maintain the correctness of shared and replicated objects



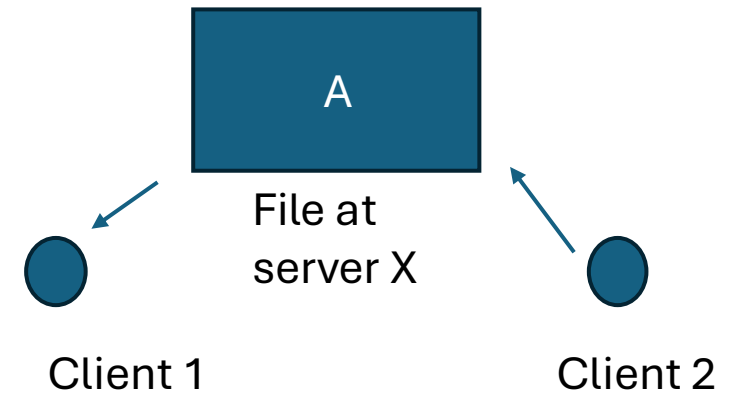
Concurrency

Concurrency

- In general, the transactions of clients access/update objects at multiple servers
 - Each transaction has a sequence of operations
- Sometimes, different transactions may interfere with each other.
At some point, they might need access to/manipulate the same object at the same time

EX:

- Two concurrent events,
READ (from Client 1), WRITE(B) (from Client 2)
- Ordering matters;
 - If READ is completed first, then
Client 1 gets A, and the file is then updated to B
 - If WRITE is completed first, then the file is updated to B, and client 1 gets B. In this case, if there is a delay in updating data at server X, then Client 1 still gets A, and later the data will be updated to B



Concurrency

- **Shared object:** may be **simultaneously accessed by multiple events**.
- A server which manages shared objects is responsible for ensuring the objects remain consistent when accessed by concurrent events.
- Such a control is called ***concurrency control***.
- This control is usually done by **locking**.
- **Extension to systems with replication:**
 - If event **T** happens before event **U** in their conflicting access to objects at one of the servers,
 - then they must be in that order at all of the servers whose objects are accessed in a conflicting manner by both **T** and **U**.

Reference

- **Chapter 7 of**
Distributed Systems, 4th Edition,
Maarten van Steen and Andrew
Tannenbaum.
- **Chapters 17 and 18 of**
Distributed systems : concepts and
design / George Coulouris ... [et al.],
Addison-Wesley (Pearson), c2012.

